TRUST-AWARE MODEL ROUTING

Aegis Agent | First-person engineering account | Source-grounded architecture

My engineering position. I implemented a rule-driven routing layer that separates specialist selection, model-tier labels, response validation and bounded fallback. I use the term trust-aware to describe the routing signals and control surfaces visible in the source code; I do not present its heuristic scores as experimentally calibrated trust measurements.

01 | Architectural boundaries

I distinguish two related but nonidentical mechanisms. TrustAwareModelRouter computes specialist candidates and cheap/expensive model labels, and exposes a separate validated route() path. AICore.dispatch() selects its specialist through its own keyword resolver, then consults the router only for the LLM model label. I do not claim that every dispatch uses the same agent-selection algorithm as decide_model().

Implementation	Verified responsibility
core/model_router.py	Rule-based specialist scores, tier labels, synthetic evaluation metadata, candidate fallback, circuit breaker and schema validation.
core/ai_core.py	Independent keyword agent resolver, cache lookup, provider completion and response metadata.
core/llm_provider.py	Offline Echo default; optional configured HTTP completion with Echo fallback.
core/agent_registry.py	Six registered specialist roles.
core/token_optimizer.py	In-memory response cache and token usage estimates.

02 | Specialist routing: my scoring rules

In decide_model(), I start all six registered specialists at zero. I add two points for each matching route keyword for Alina, Kian, Bita and Aylin. I add explicit privacy, cost and latency modifiers. If all scores remain zero, I choose Alina. A preferred specialist is accepted only if its name exists in the registered role map.

Signal in task	Code-level routing effect
Plan / strategy / roadmap	Adds score to Alina.
Deploy / execute / runtime	Adds score to Kian.
Analyze / risk / context	Adds score to Bita.
Validate / audit / test	Adds score to Aylin.
Privacy or secure	Adds score to Bita and Aylin.
Cost or budget	Adds score to Alina and Aylin.

Signal in task	Code-level routing effect
Latency or fast	Adds score to Kian.

Although Ahmad and Amin are registered, the ROUTE_KEYWORDS table in this method does not provide dedicated keyword groups for them. I therefore do not claim equal automatic selection coverage across all six specialists in this particular algorithm.

03 | My model-tier decision

I label the expensive tier aegis-expensive when task text exceeds 800 characters or includes one of the configured trigger strings: architecture, security-review, incident, settlement, multi-step, design or refactor. Otherwise I label it aegis-cheap. This is a deterministic rule, not a measured inference-cost optimization model.

Input example	Tier result	Reason
Prepare a weekly brief	aegis-cheap	No expensive trigger.
Review the architecture	aegis-expensive	Architecture trigger.
Long request above 800 characters	aegis-expensive	Length threshold.

The model names are labels passed to the configured provider. I do not equate them with two verified paid cloud models. In the default configuration, EchoProvider returns a deterministic offline response that includes the selected label.

04 | Trust signals: what I actually calculate

My evaluate() function returns cost, latency, privacy and capability scores, plus their arithmetic mean as confidence. These numbers are fixed conditional values determined by keyword presence and whether a specialist name is registered. They are not derived from measured latency, live price quotes, privacy audits or historical model accuracy.

Signal	Rule-based output
Cost score	0.90 for cost/budget text; otherwise 0.80.
Latency score	0.95 for latency/fast text; otherwise 0.85.
Privacy score	0.95 for privacy/secure text; otherwise 0.80.
Capability score	0.90 for registered selected specialist; otherwise 0.75.
Confidence	Arithmetic mean of the four score values.

I describe these as heuristic metadata, not as empirical probability, assurance or a compliance guarantee. A 0.95 privacy score does not prove privacy preservation.

05 | Validated routing and fallback

My route() method builds an ordered candidate list: selected specialist first, followed by other registered specialists. Before each candidate, I consult the circuit breaker. For allowed candidates, I build a response payload and validate it against AgentResponse with Pydantic. I retry schema-validation failures up to max_retries + 1 attempts, then consider another candidate. Runtime exceptions are recorded and end the current candidate loop.

On success, I reset the candidate circuit breaker and append a route-success ledger event. On failure, I record a breaker failure and append failure or exception evidence. An open circuit records an event and a scorecard rollback. These are implemented control paths; I have not independently measured recovery rates or demonstrated distributed resilience.

06 | A critical integration distinction

In _build_payload(), when no factory is supplied, I create a new AICore and call dispatch(task). That dispatch chooses its own specialist via resolve_agent_name(), rather than necessarily executing the fallback candidate being iterated by route(). The returned payload uses the actual dispatched agent name. Therefore I do not claim that candidate fallback reliably forces a different specialist without a caller-provided factory or additional integration work.

I also do not claim that the public operator cognitive-twin propose/approve flow necessarily calls this validated router. Source-level existence of a component is not proof that every user-facing path traverses it.

07 | Provider behavior and cost boundaries

My provider abstraction defaults to EchoProvider, with no network completion. An optional HTTP provider requires explicit configuration and usable credentials. When an HTTP request fails, its completion method falls back to Echo. I do not call that a successful cloud-model response, nor do I treat a chosen expensive label as proof of greater model quality.

The token optimizer caches normalized task strings in process memory and estimates tokens from character counts. It is not a billing-grade metering service, and its state is not a persistent cross-process cache by default. I keep measured provider token fields separate from local estimates in any performance claim.

08 | Worked decision traces

Task	decide_model()	decide_llm_model()
Plan a product roadmap	Alina	aegis-cheap
Validate the architecture	Aylin	aegis-expensive
Review privacy risk	Bita	aegis-cheap
Deploy and monitor service	Kian	aegis-cheap

These examples are reasoned from the source rules, not presented as live execution traces. In ambiguous multi-keyword tasks, additive scores and insertion-order tie behavior affect selection; I do not claim semantic intent understanding beyond those rules.

09 | Security, correctness and failure questions

Routing trust: the scoring function uses substring checks; a quoted or negated keyword can still affect the result. Fallback: candidate iteration does not necessarily change the actual AICore-dispatched specialist. Isolation: the router evidence events use tenant_id=system; that alone does not demonstrate tenant-specific provenance. Model identity: provider-side acceptance of the label depends on configuration. Validation: a schema-valid response is not automatically factually correct or safe.

I would require end-to-end tests for candidate forcing, tenant propagation, cache correctness across agent selection, model-tier/provider mapping, circuit-breaker recovery and response safety before making stronger production guarantees.

10 | Evidence and verification status

I reviewed the public source implementations of core/model_router.py, core/ai_core.py, core/llm_provider.py, core/token_optimizer.py and core/agent_registry.py. I did not execute the complete test suite, benchmark live model selection or certify production behavior for this publication. Every example above is an illustrative derivation of inspected control flow.

11 | Claims I make and withhold

I can substantiate: deterministic specialist scoring, cheap/expensive tier labels, heuristic metadata, an explicit schema-validated route path, candidate-level breaker checks, bounded retries, ledger events and a default offline provider.

I do not claim: statistically calibrated trust scores, validated privacy guarantees, adaptive reinforcement-based routing, universal integration into the operator flow, forced specialist failover in the default factory path, proven latency savings or two independently deployed commercial model tiers.

My architectural conclusion. I treat this routing implementation as an inspectable coordination layer with useful explicit controls and clearly bounded semantics. I distinguish the mechanisms already implemented from the stronger reliability and model-governance guarantees that would require additional integration and operational evidence.

12 | Public source references

Repository: github.com/aminazimi42-coder/aegis-agent. Primary files: core/model_router.py, core/ai_core.py, core/llm_provider.py, core/token_optimizer.py and core/agent_registry.py. I made no GitHub repository changes in preparing this publication.

Authorship: Amin Azimi | AI Architect | Azimi Innovation Lab. My first-person engineering statements are bounded by the public source reviewed, not an independent certification of runtime performance.